Quantum Order Finding for $N = 21$

Carolina Amorim, Owen Barnes, Summer Malik

June 28, 2026

1 Overview

Shor's algorithm reduces integer factorization to *order finding*. To factor $N = 21$, we pick a base a coprime to N and find the *order* r: the smallest positive integer such that $a^r \equiv 1 \pmod{21}$. Once r is known (and is even), the factors are recovered classically as $\gcd\big(a^{r/2} \pm 1,\ N\big)$.

The quantum part of the algorithm finds r using *quantum phase estimation* (QPE). Our circuit uses two registers:

- a **counting register** (n qubits) that reads out the phase,
- a **work register** (5 qubits, since $\lceil \log_2 21 \rceil = 5$) that holds the residues mod 21.

2 Circuit construction

The order-finding circuit is built in three stages:

1. Place the counting register in uniform superposition with Hadamard gates, and initialize the work register to $|1\rangle$.
2. Apply controlled modular multiplication: for each counting qubit k, a controlled gate maps $|y\rangle \mapsto |a^{2^k} y \bmod 21\rangle$. This encodes the periodic function whose period is of order r.
3. Apply the inverse Quantum Fourier Transform to the counting register and measure it. The measured bitstrings concentrate at values $k \cdot 2^n / r$, from which r is recovered using the continued-fraction expansion.

The controlled multiplication is implemented as an explicit permutation matrix that realizes the map $y \mapsto a^{2^k} y \bmod 21$, which is exact on a simulator. The inverse QFT is implemented directly from Hadamard and controlled-phase gates.

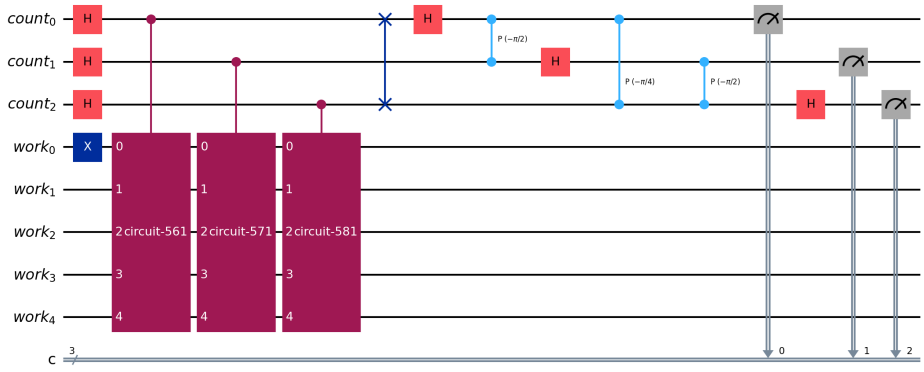

Figure 1: Order-finding circuit for $N = 21$ with base $a = 8$, shown with three counting qubits for readability. From left to right: Hadamard gates on the counting register, the work register initialized to $|1\rangle$, the controlled modular-multiplication blocks, the inverse QFT, and the final measurements. The full simulation uses additional counting qubits for finer phase resolution.

3 Code walkthrough

Imports and constants

```
N = 21
N_WORK = 5
```

`N` is the number to factor (in this case, 21). `N_WORK` is the size of the work register: since the largest residue modulo 21 is 20, and $\lceil \log_2 21 \rceil = 5$, five qubits are required to store any value modulo 21.

The inverse Quantum Fourier Transform

```
def inverse_qft(circuit, qubits):
    n = len(qubits)
    for i in range(n // 2):
        circuit.swap(qubits[i], qubits[n - 1 - i])
    for j in range(n):
        for m in range(j):
            circuit.cp(-pi / (2 ** (j - m)), qubits[m], qubits[j])
        circuit.h(qubits[j])
    return circuit
```

The inverse QFT converts the phase, which is encoded in the relative phases of the counting qubits, into a binary number that can be measured. It is built from two ingredients. The first loop performs a sequence of `swap` gates that reverses the qubit order; this is the standard bit-reversal step of the QFT. The second loop applies, for each qubit, a set of controlled-phase rotations (`cp`) followed by a Hadamard gate. The rotation angles are negative, $-\pi/2^{(j-m)}$, which is what distinguishes the *inverse* transform from the forward QFT.

Controlled modular multiplication

```
def c_amod21(a, power):
    mult = pow(a, 2 ** power, N)
    M = np.zeros((32, 32))
    for y in range(32):
        fy = (mult * y) % N if y < N else y
        M[fy, y] = 1.0
    U = QuantumCircuit(N_WORK)
    U.unitary(Operator(M), range(N_WORK), label=f"{a}^{2**power} mod21")
    return U.to_gate().control()
```

This is the heart of the algorithm: the operation whose period encodes the order r. It builds the gate that maps a basis state $|y\rangle$ to $|a^{2^{\text{power}}} \cdot y \bmod 21\rangle$.

The variable `mult` is the multiplier $a^{2^{\text{power}}} \bmod 21$; the powers of two appear because phase estimation requires the controlled operation to be applied 2^k times for the k-th counting qubit, and repeated multiplication is equivalent to multiplying by this power directly. The matrix `M` is a 32×32 permutation matrix (the work register spans $2^5 = 32$ basis states): for each input y, a single 1 is placed in the row $f(y)$, where $f(y) = \text{mult} \cdot y \bmod 21$ for valid residues and $f(y) = y$ otherwise. This matrix is loaded into the circuit as a unitary and then converted into a *controlled* gate with `.control()`, so it acts only when the corresponding counting qubit is $|1\rangle$.

Assembling the order-finding circuit

```
def build_order_finding_circuit(a=8, n_count=3):
    count = QuantumRegister(n_count, "count")
    work  = QuantumRegister(N_WORK, "work")
    cr    = ClassicalRegister(n_count, "c")
    qc = QuantumCircuit(count, work, cr)
    qc.h(count)
```

```
    qc.x(work[0])
    for k in range(n_count):
        qc.append(c_amod21(a, k), [count[k]] + list(work))
    inverse_qft(qc, list(count))
    qc.measure(count, cr)
    return qc
```

This function combines the pieces into the full phase-estimation circuit. It creates the counting register, the work register, and a classical register for the measurement results. The Hadamard gates (`qc.h(count)`) place the counting register into a uniform superposition, and `qc.x(work[0])` initializes the work register to $|1\rangle$, the starting value for the modular multiplications. The loop appends one controlled multiplication per counting qubit, each controlled by qubit k and acting on the work register. The inverse QFT is then applied to the counting register, and finally that register is measured into the classical bits.

Running the circuit

```
def run_circuit_and_get_counts(circuit, backend, shots=1000):
    pm = generate_preset_pass_manager(backend=backend,
                                      optimization_level=1)
    isa_circuit = pm.run(circuit)
    sampler = Sampler(mode=backend)
    job = sampler.run([isa_circuit], shots=shots)
    result = job.result()
    return result[0].data.c.get_counts()
```

This function executes the circuit. The *pass manager* transpiles the circuit into gates supported by the chosen backend; the same code therefore runs unchanged on a local simulator or on real IBM hardware. The `Sampler` runs the circuit `shots` times and returns a dictionary of measured bitstrings and their frequencies (the `counts`).

Recovering the order and the factors

```
def recover_order(counts, a=8, n_count=10):
    for bits, _ in sorted(counts.items(),
                          key=lambda kv: -kv[1]):
        phase = int(bits, 2) / (2 ** n_count)
        if phase == 0:
```

```
            continue
        for cap in range(2, N + 1):
            r = Fraction(phase).limit_denominator(cap).denominator
            if r > 0 and pow(a, r, N) == 1:
                return r
    return None
```

This is the classical post-processing step. Each measured bitstring is converted to a phase by interpreting it as a binary fraction (phase = measured/2^n). The zero outcome is skipped because it carries no information about the period. The continued-fraction expansion, implemented by `Fraction(...).limit_denominator`, finds the best rational approximation of the phase with a bounded denominator; that denominator is the candidate order r. The candidate is accepted only if it truly satisfies $a^r \equiv 1 \pmod{21}$. Once r is found, the factors follow from $\gcd(a^{r/2} \pm 1, 21)$.

4 Comparison of two bases

We compare two valid bases, $a = 8$ and $a = 2$. Both correctly factor $21 = 3 \times 7$, but they differ in efficiency because they have different orders. The number of peaks in the measured spectrum equals the order r.

Metric	$a = 8$	$a = 2$
Order r	2	6
Factors found	3×7	3×7
Counting qubits	3	5
Total qubits	8	10
Transpiled depth	1240	6199
Distinct outcomes	2	32

Table 1: Comparison of two coprime bases for factoring $N = 21$. Both succeed; $a = 8$ does so with substantially fewer resources.

5 Discussion of results

Both bases recover a valid order and factor $N = 21$, but the choice has a clear effect on efficiency. The base $a = 8$ has order $r = 2$, the smallest non-trivial period. As a result, it requires fewer counting qubits, produces a significantly

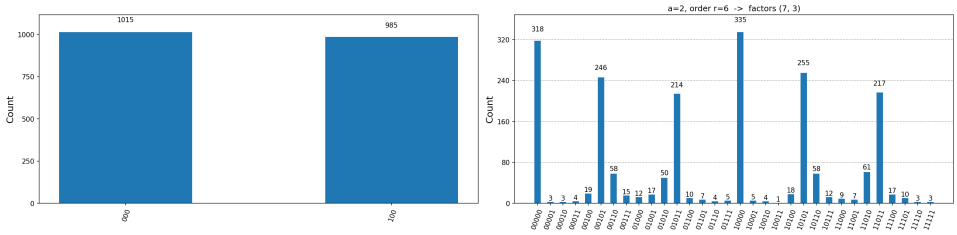

Figure 2: Measured phase distributions. Left: $a = 8$ (order 2), two peaks. Right: $a = 2$ (order 6), the full spectrum.

smaller circuit (depth 1240 versus 6199), and yields an unambiguous two-peak measurement. By contrast, $a = 2$ has order $r = 6$, and produces the full six-peak phase spectrum, which is more illustrative of the period-finding mechanism, but considerably less efficient and more costly.

The shallower circuit of the $a = 8$ base is especially advantageous on real quantum hardware, where shorter circuits accumulate less noise. We therefore regard $a = 8$ as the more efficient choice for a hardware demonstration, while $a = 2$ remains useful for visualizing the complete spectrum produced by a quantum phase estimation.